

TF-IDF e Similaridade de Cosseno

Aula 12 – Inteligência Computacional

Eduardo Corrêa Gonçalves

02/10/2026

Sumário

Introdução

TF-IDF

Apresentando o TF e o IDF

Vantagens e Desvantagens

Similaridade de Cosseno

Introdução (1/3)

Relembrando a aula sobre o k-Médias

- Na aula sobre o k-Médias estudamos como **medidas de distância** podem ser empregadas para avaliar a **dissimilaridade** entre pares de objetos.
- Os objetos são representados como **vetores numéricos** no espaço n -dimensional. Quanto mais distantes, mais dissimilares.
- Foram apresentados exemplos em que empregamos a medida da Distância Euclidiana (DE).

s				
0	2	1	2	0

t				
1	2	0	1	1

$$DE(s, t) = \sqrt{(0 - 1)^2 + (2 - 2)^2 + (1 - 0)^2 + (2 - 1)^2 + (0 - 1)^2} = 2$$

Introdução (2/3)

O ponto fraco da Distância Euclidiana

- A DE funciona bem em muitos problemas.
- Porém costuma “**falhar**” em aplicações envolvendo vetores **esparcos**.
 - Isso é, vetores que possuem zeros na maioria de suas posições.
- Esse caso é muito comum em aplicações que envolvem **dados não estruturados**, como **textos** e **imagens**.

	[0]	[1]	[2]	[3]	[4]	[...]	[49998]	[49999]
v	0	2	0	0	0	...	0	0

Introdução (3/3)

Aula de Hoje: Similaridade do Cosseno

- Na aula de hoje, vamos examinar a medida de **similaridade do cosseno**.
 - Medida de similaridade padrão para vetores esparsos
- O caso prático do Notebook será baseado em um exemplo que examina a similaridade entre frases.
- Mas antes do cosseno, mostraremos uma técnica de AM muito utilizada para transformar textos em vetores: **TF-IDF**

Bag of Words

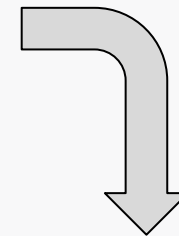
- Matriz Bag of Words (BoW)

- Formato **mais básico** para representar um conjunto de textos de forma numérica (é usada pelo LDA e outros algoritmos).
 - Cada **linha** corresponde a um **documento** i .
 - Cada **coluna** corresponde a um **termo** j (i.e., um token específico).
 - Cada **célula** vai registrar a **frequência** do termo j no documento i .

["gatinho", "vovó", "fez", "miau"]

["vovó", "viu", "uva"]

["arara", "araraquara", "arara", "vovó"]



	arara	araraquara	fez	gatinho	miau	uva	viu	vovó
[D1]	0	0	1	1	1	0	0	1
[D2]	0	0	0	0	0	1	1	1
[D3]	2	1	0	0	0	0	0	1

TF-IDF (1/9)

- Desvantagem da Matriz Bag of Words
 - Muitas vezes, não consegue atribuir maior peso para as palavras que, de fato, melhor caracterizam um documento.
 - Veja esse frase: “*No Mineirão, o jogo foi ótimo*”
 - Todas as palavras têm frequência 1
 - Porém, “*Mineirão*” (principalmente) e “*jogo*” são as que realmente caracterizam que o texto é sobre futebol.
 - Por isso, merecem um peso maior !!!
 - Mas como conseguir isso? Veja o cenário do próximo slide.

TF-IDF (2/9)

- CENÁRIO:
 - Seja um **corpus** em que os textos são **notícias de jornais**.
 - Suponha que deseja-se identificar os textos sobre o tema “**futebol**”.
 - Faz sentido supor que:
 - A palavra “**Mineirão**” aparecerá em um razoável número de textos de futebol ... Porém, sua ocorrência será muito rara em textos de outros assuntos (ex.: economia, artes).
 - Já a palavra “**ótimo**” será mais frequente no corpus, aparecendo em textos de diferentes temas.
 - **Ou seja:** “**Mineirão**” tem frequência baixa no corpus, mas é boa para caracterizar textos sobre futebol.
- Agora, precisamos modelar esse fenômeno na matriz.

TF-IDF (3/9)

- É exatamente para isso que existe a **medida TF-IDF** (*Term Frequency – Inverse Document Frequency*).
- Seu objetivo é **medir o quanto a ocorrência de um determinado termo está concentrada em poucos documentos**.
 - Ou seja, em documentos que abordam algum tema específico.
- Ela combina duas medidas: **TF** (*Term Frequency*) e **IDF** (*Inverse Document Frequency*).
- Vamos conhecê-las a seguir.

TF-IDF (4/9)

- TF (*Term Frequency*)

- Seja:

- $f(d_i, t_j)$: frequência do termo t_j no documento d_i .
 - f_{max} : frequência do termo com o maior número de ocorrências no documento d_i .

- $TF(d_i, t_j)$ é calculado pela seguinte fórmula:

$$TF_{(d_i, t_j)} = \frac{f_{(d_i, t_j)}}{f_{max}}$$

- Quanto mais frequente o termo em um documento, maior será o seu valor de TF nesse documento específico.
 - O termo mais frequente do documento receberá o valor de 1 para TF enquanto os outros receberão valores abaixo de 1.

TF-IDF (5/9)

- **IDF (*Inverse Document Frequency*)**
 - Seja:
 - n : total de documentos do corpus
 - n_t : total de documentos em que o termo t aparece
 - O valor de $IDF_{(t)}$ é calculado pela seguinte fórmula:

$$IDF_{(t)} = \log_2 \left(\frac{n}{n_t} \right)$$

- Quanto mais comum o termo no corpus, menor será o seu IDF

TF-IDF (6/9)

- **IDF - Exemplo de Cálculo**

- Considere um corpus formado por 100.000 notícias, onde:
 - A palavra “**Mineirão**” ocorre em apenas 10 documentos;
 - A palavra “**ótimo**” ocorre em 2.500 documentos.
- $IDF_{(mineirão)} = \log_2(100.000 / 10) = 13,29$
- $IDF_{(ótimo)} = \log_2(100.000 / 2.500) = 5,32$

TF-IDF (7/9)

- **TF-IDF – é uma combinação das duas**
 - A medida TF-IDF para um termo t_j em um documento d_i é dada por:

$$\text{TF-IDF}(d_i, t_j) = \text{TF}(d_i, t_j) \times \text{IDF}(t_j)$$

- Considera, ao mesmo tempo, o quanto o termo é frequente no documento e o quanto ele é raro no corpus como um todo.
- Assim, espera-se que os termos com maior valor de TF-IDF em um documento sejam os que melhor o caracterizam.

TF-IDF (8/9)

- **Exemplo de Cálculo (1/2)**

- Considere um corpus formado por **100.000 notícias**, onde:
 - A palavra “**Mineirão**” ocorre em apenas 10 documentos;
 - A palavra “**ótimo**” ocorre em 2.500 documentos.
 - A palavra “**jogo**” também ocorre em 2.500 documentos.
- Seja um documento específico d , sobre futebol, onde:
 - “**Mineirão**” ocorre 2 vezes;
 - “**ótimo**” ocorre 2 vezes;
 - “**jogo**” ocorre 4 vezes;
 - “**jogo**” é a palavra que ocorre o maior número de vezes em d .
- Como calcular o valor da medida TF-IDF para as palavras “**Mineirão**”, “**ótimo**” e “**jogo**” no documento d ?

TF-IDF (9/9)

- **Exemplo de Cálculo (2/2)**

- Primeiro calculamos o TF dos termos “mineirão”, “ótimo” e “jogo” no documento d :

- $TF_{(d, \text{mineirão})} = 2 / 4 = 0,50$ $TF_{(d, \text{ótimo})} = 2 / 4 = 0,50$ $TF_{(d, \text{jogo})} = 4 / 4 = 1,00$

- Em seguida, calculamos o IDF de “mineirão”, “ótimo” e “jogo” no corpus:

- $IDF_{(\text{mineirão})} = \log_2(100.000 / 10) = 13,29$
- $IDF_{(\text{ótimo})} = \log_2(100.000 / 2.500) = 5,32$
- $IDF_{(\text{jogo})} = \log_2(100.000 / 2.500) = 5,32$

- Agora é possível calcular o TF-IDF dos 3 termos em d :

- $TF-IDF_{(d, \text{mineirão})} = 0,50 \times 13,29 = \underline{\underline{6,65}}$
- $TF-IDF_{(d, \text{ótimo})} = 0,50 \times 5,32 = \underline{\underline{2,66}}$
- $TF-IDF_{(d, \text{jogo})} = 1,00 \times 5,32 = \underline{\underline{5,32}}$

- Veja que a palavra “mineirão” fica com maior peso no documento d .

Prós e Contras da DTM TF-IDF (1/2)

- VANTAGENS da DTM TF-IDF:

- ☺ Ela é simples.

- ☹ Ela é uma matriz de reais.

- Sendo assim, **serve** como conjunto de dados de entrada para praticamente **qualquer algoritmo de AM ou IC** (classificação, otimização, *clustering* etc.)

Prós e Contras da DTM TF-IDF (2/2)

- **DESVANTAGENS** da DTM TF-IDF:

☹ Os vetores numéricos ficam muito grandes e esparsos

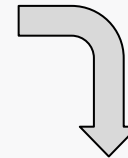
O vetor de cada documento é uma linha da matriz. As colunas são as palavras do vocabulário, que podem ser dezenas/centenas de milhares !!

Os vetores são esparsos, pois a maioria dos documentos utiliza um subconjunto pequeno do vocabulário

"O gatinho da vovó fez "miau""

"Vovó viu a uva"

"A arara de Araraquara é a arara da vovó"



	arara	araraquara	fez	gatinho	miau	uva	viu	vovó
[D1]	0.00	0.00	0.56	0.56	0.56	0.00	0.00	0.27
[D2]	0.00	0.00	0.00	0.00	0.00	0.67	0.67	0.32
[D3]	0.87	0.44	0.00	0.00	0.00	0.00	0.00	0.21

Similaridade de Cosseno (1/8)

- **Cosseno x Distância Euclidiana**
 - Conforme visto na aula passada, podemos medir dissimilaridade com a Distância Euclidiana.

Medida simples, intuitiva e largamente utilizada em problema de similaridade.
 - Porém, não funciona bem em situações em que temos **vetores esparsos**.

É exatamente o caso das DTMs TF-IDF, que acabamos de conhecer.

Similaridade de Cosseno (2/8)

- Cosseno x Distância Euclidiana

- **Exemplo:** considere 3 vetores TF-IDF
- $v1 = [0, 0, 0.2]$ → *doc não contém palavras p1 e p2 e contém p3 c/ TF-IDF=0.2*
- $v2 = [0, 0, 0.8]$ → *doc não contém palavras p1 e p2 e contém p3 c/ TF-IDF=0.8*
- $v3 = [0, 0, 0]$ → *doc não contém p1, p2, p3*
- Está claro que v1 possui mais similaridade com v2 do que com v3.
- Porém, a Dist. Euclid. entre v1 e v2 é maior do que a entre v1 e v3 !!!

$$DE(v1, v2) = \sqrt{(0 - 0)^2 + (0 - 0)^2 + (0.2 - 0.8)^2} = 0.6$$

$$DE(v1, v3) = \sqrt{(0 - 0)^2 + (0 - 0)^2 + (0.2 - 0.0)^2} = 0.2$$

Similaridade de Cosseno (3/8)

- **Cosseno**

- A medida do cosseno pode ser usada em situações deste tipo.
- Ela é a mais usada para similaridade entre textos. É computada por:

$$\text{Cos}(s, t) = \frac{s \cdot t}{\|s\| \|t\|}$$

- $\|s\|$ é a norma Euclidiana, computada por: (*analogamente para $\|t\|$*)

$$\|s\| = \sqrt{s_{[1]}^2 + s_{[2]}^2 + \dots + s_{[n]}^2}$$

- $s \cdot t$ é o produto escalar dos vetores:

$$s \cdot t = s_{[1]}t_{[1]} + s_{[2]}t_{[2]} + \dots + s_{[n]}t_{[n]}$$

Similaridade de Cosseno (4/8)

- Por que o produto escalar no numerador?
 - Ele atua como uma medida de similaridade porque tende a ser alto apenas quando s e t têm valores grandes nas mesmas posições.

$$s \cdot t = s_{[1]}t_{[1]} + s_{[2]}t_{[2]} + \cdots + s_{[n]}t_{[n]}$$

- Além disso, **vetores ortogonais** (com zeros em dimensões diferentes), terão um produto escalar de 0.
 - Isso representa bem a forte dissimilaridade entre eles.

$$\text{Cos}(s, t) = \frac{s \cdot t}{\|s\| \|t\|}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

Similaridade de Cosseno (5/8)

- Por que o produto das normas no denominador ?
 - Primeiro para normalizar o valor para a faixa entre 0.0 e 1.0 (correspondendo automaticamente ao **cosseno do ângulo entre os vetores**).

$$||s|| = \sqrt{s_{[1]}^2 + s_{[2]}^2 + \dots + s_{[n]}^2}$$

$$||t|| = \sqrt{t_{[1]}^2 + t_{[2]}^2 + \dots + t_{[n]}^2}$$

- Também porque foi percebido na prática que o cosseno funciona melhor do que usar apenas o produto escalar (detalhes em Jurafsky e Martin (2025)).

$$Cos(s, t) = \frac{s \cdot t}{||s|| ||t||}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

Similaridade de Cosseno (6/8)

- Como funciona?
 - Quanto menor o ângulo entre os vetores, mais similares eles são.
 - O que interessa é o ângulo, não o tamanho dos vetores.
 - Essa característica é boa para: (i) comparar textos; (ii) comparar vetores esparsos.
 - Para vetores representando textos, resultará sempre em um valor entre 0 e 1.
 - $\text{Cos}(s, t) \approx 1$, se os vetores são muito similares ($\cos(0^\circ) = 1$)
 - $\text{Cos}(s, t) \approx 0$, se os vetores são muito dissimilares ($\cos(90^\circ) = 0$)

$$\text{Cos}(s, t) = \frac{s \cdot t}{\|s\| \|t\|}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

Similaridade de Cosseno (7/8)

- Exemplo: cálculo da similaridade de cosseno entre s e t

s				
0	2	1	2	0

t				
1	2	0	1	1

$$\cos(s, t) = \frac{s \cdot t}{\|s\| \cdot \|t\|}$$

$$\begin{aligned}\cos(v_1, v_2) &= \frac{(0 \times 1) + (2 \times 2) + (1 \times 0) + (2 \times 1) + (0 \times 1)}{\sqrt{0^2 + 2^2 + 1^2 + 2^2 + 0^2} \times \sqrt{1^2 + 2^2 + 0^2 + 1^2 + 1^2}} \\ &= \frac{6}{\sqrt{9} + \sqrt{7}} = \mathbf{0.76}\end{aligned}$$

Similaridade do Cosseno (8/8)

- O Cosseno é uma medida de similaridade de fato.
- A definição de medida de similaridade é essa abaixo:
 - Uma **medida de similaridade S** funciona da seguinte forma:
 1. $S(s, t) = 1$ se $s = t$;
 2. $S(s, t) \approx 1$ quando s é muito parecido com t ;
 3. $S(s, t) \approx 0$ quando s é muito diferente de t .

Link para o notebook com o código

- Notebook com exemplo de similaridade entre frases na sklearn – usando TF-IDF e Cosseno:
 - https://colab.research.google.com/drive/1ayYvKzSH8-63WnveOu6ker_duzta2IXG?usp=sharing

Principais conceitos apresentados

- **Vetor esperso:** vetor que possui o valor zero na maioria de suas posições.
- **Corpus:** conjunto de textos coletados e organizados com algum propósito. Pode ser entendido como uma base de dados em que cada objeto corresponde a um documento (texto).
- **Documento:** no jargão do AM, qualquer texto é chamado de documento (mesmo que seja uma frase ou um post em uma rede social).
- **Tokenização:** processo de separar as palavras, números e pontuações de um documento. Cada item separado é chamado de *token*.
- **Vocabulário:** conjunto contendo todos os termos distintos que aparecem em um corpus.
- **Matriz Termo-Documento (*Term-Document Matrix* - *DTM*):** matriz que representa os documentos de um corpus como vetores de tamanho fixo. Nessa matriz, cada linha é um documento do corpus e cada coluna uma palavra do vocabulário.
- **TF-IDF:** medida utilizada na DTM para representar o peso de um termo em um dado documento, onde o peso reflete a importância da palavra para caracterizar o documento.
- **Similaridade de Cosseno:** medida de similaridade que corresponde ao valor do cosseno entre dois vetores numéricos. É a mais utilizada por aplicações que lidam com vetores esparsos.

Bônus: TF-IDF na scikit-learn (1/4)

- Algumas especificidades do TF-IDF implementado na scikit-learn
 - Algumas ferramentas usam **versões modificadas** da TF-IDF.
 - Por exemplo, na **scikit-learn**:
 - (1) O TF é apenas a frequência do termo no documento (sem dividir pelo número de ocorrências do termo mais frequente)
 - (2) A fórmula do IDF também é ligeiramente diferente (há 2 opções, sendo que a da esquerda é a *default* e a da direita a alternativa)

$$IDF_{(t)} = \log \left(\frac{n}{n_t} \right) + 1 \quad \text{ou} \quad IDF_{(t)} = \log \left(\frac{1 + n}{1 + n_t} \right) + 1$$

- (3) O vetor v contendo os escores TF-IDF para m termos de um documento é padronizado para valores entre 0 e 1 utilizando a norma Euclidiana (l2)

$$v_{norm} = \frac{v}{\sqrt{(v_1^2 + v_2^2 + \dots + v_m^2)}}$$

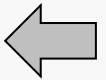
Bônus: TF-IDF na scikit-learn (2/4)

- Exemplo na scikit-learn: **Calculo do TF-IDF do documento D_1 :**

- PASSO 1:** calcula o TF-IDF

- $IDF_{(t_1)} = \log(6 / 6) + 1 = 1,00$
- $IDF_{(t_2)} = \log(6 / 1) + 1 = 2,79$
- $IDF_{(t_3)} = \log(6 / 2) + 1 = 2,10$
- $TF_IDF_{(D_1, t_1)} = 3 \times 1,00 = \underline{3,00}$
- $TF_IDF_{(D_1, t_2)} = 0 \times 2,79 = \underline{0,00}$
- $TF_IDF_{(D_1, t_3)} = 1 \times 2,10 = \underline{2,10}$

	t_1	t_2	t_3
$[D_1]$	3	0	1
$[D_2]$	2	0	0
$[D_3]$	3	0	0
$[D_4]$	4	0	0
$[D_5]$	3	2	0
$[D_6]$	3	0	2



- PASSO 2:** normaliza os valores

$$TF_IDF_{(D_1, norm)} = \frac{[3,00, 0,00, 2,10]}{\sqrt{(3,00^2 + 0,00^2 + 2,10^2)}} = [0,819, 0,00, 0,573]$$

Bônus: TF-IDF na scikit-learn (3/4)

- **Tokenização na scikit-learn**
 - É importante ainda observar que a scikit-learn adota os seguintes **passos para fazer a tokenização** dos textos:
 1. **Converter** o texto para **minúsculo**
 2. **Remover pontuações**
 3. Considerar como **tokens** as **sequências com 2 ou mais caracteres** formadas por **letras**, **números** ou **underscore**
 - Isso vale tanto para a matriz BoW como para a TF-IDF
 - É possível mudar esse comportamento padrão... mas se você não mexer em nenhum parâmetro, funcionará do jeito acima.

Bônus: TF-IDF na scikit-learn (4/4)

- Exemplo: veja os tokens gerados pela scikit... e também a DTM TF-IDF produzida para esse único documento

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

texto = ["HOJE TEM FEIRA NA PRAÇA DA LIBERDADE. A batata-doce
custa R$ 9,50. Quem vai à feira comprar?"]

tf_vect = TfidfVectorizer()
dtm = tf_vect.fit_transform(texto).toarray()
tokens = tf_vect.get_feature_names_out()

print('tokens:', tokens)
print('\ndtm:')
print(dtm)
```

```
tokens: ['50' 'batata' 'comprar' 'custa' 'da' 'doce' 'feira'
'hoje' 'liberdade' 'na' 'praça' 'quem' 'tem' 'vai']
```

```
dtm:
[[0.24253563 0.24253563 0.24253563 0.24253563 0.24253563
0.24253563 0.48507125 0.24253563 0.24253563 0.24253563
0.24253563 0.24253563 0.24253563 0.24253563]]
```

Referências

- JURAFSKY, D.; MARTIN, J. H. Vector Semantics and Embeddings. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. *draft* [s.l.] : [s.n.], 2025. cap 6, p. 102-117. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 28 mar. 2025.
- LESCOVEC, J.; RAJARAMAN, A.; ULLMAN, J. Importance of Words in Documents. *In*: LESCOVEC, J.; RAJARAMAN, A.; ULLMAN, J. **Mining of Massive Datasets**. 3. ed. [s.l.] : [s.n.], 2014. Disponível em: <http://www.mmds.org/>. Acesso em: 06 jan. 2025. cap 1, p. 10 - 11.
 - MOREIRA, P. Recuperação da Informação. *In*: CASELI, H. M.; NUNES, M. G. V. (org.). **Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português**. 3. ed. [s.l.] : BPLN, 2024. Disponível em: <https://brasileiraspln.com/livro-pln/3a-edicao/parte-aplicacoes/cap-ir/cap-ir.html>. Acesso em: 06 jan. 2025. cap 21, p. 7 - 11.
 - PARRISH, A. **Understanding word vectors**. Github.com, 2018. Disponível em: < <https://gist.github.com/aparrish/2f562e3737544cf29aaf1af30362f469> >. Acesso em: 05 dez.
 - Text Feature Extraction. *In*: scikit-learn documentation. Disponível em: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction. Acesso em: 06 jan. 2025.